

```
-- =====  
  
-- HOMEWORK: Indexes & Stored Procedures  
  
-- Topic : SQL Indexes + Stored Procedures  
  
-- Level : Beginner to Intermediate  
  
-- ===== Assignment_6 =====  
  
Name : NAHEED DEEDAR  
  
ID : 801925  
  
Assignment_6  
  
-- =====  
  
-- PART A: INDEXES  
  
-- =====  
  
-- Q1.  
  
-- Write a query to create a non-clustered index on the  
  
-- last_name column of sales.customers.  
  
-- Then write a SELECT statement that would benefit from it.  
  
-- Hint: Think about which queries filter by last name.
```

The screenshot shows a SQL query editor window titled "SQLQuery1.sql...ujitsu (62)*". The script contains the following SQL code:

```
4  -- last_name column of sales.customers.
5  -- Then write a SELECT statement that would benefit from it.
6  -- Hint: Think about which queries filter by last name.
7
8  -- Create Non-Clustered Index
9  CREATE NONCLUSTERED INDEX IX_Customers_LastName
10 ON sales.customers(last_name);
11
12
13 SELECT last_name FROM sales.customers;
14
15 SELECT
16     customer_id,
17     first_name,
18     last_name,
19     email
20 FROM sales.customers
21 WHERE last_name = 'Young';
```

Below the script, the "Results" tab is active, displaying a table with 2 rows and 5 columns: customer_id, first_name, last_name, and email. The data is as follows:

	customer_id	first_name	last_name	email
1	498	Edda	Young	edda.young@gmail.com
2	543	Jasmin	Young	jasmin.young@gmail.com

The status bar at the bottom indicates "Query executed successfully." and shows the system path "DESKTOP-26P7AE7 (17.0 RTM) | DESKTOP-26P7AE7\ujitsu... | BikeS".

-- Q2.

-- Create a composite index on sales.orders using

-- customer_id and order_date.

-- Write a query that filters on both columns and benefits

-- from this index.

-- Hint: Composite indexes work best when you filter on both columns.

```
1-SQLQuery1....ujitsu (62))*
4  -- customer_id and order_date.
5  -- Write a query that filters on both columns and benefits
6  -- from this index.
7  -- Hint: Composite indexes work best when you filter on both columns.
8
9  CREATE NONCLUSTERED INDEX IX_Orders_CustomerID_OrderDate
10 ON sales.orders(customer_id, order_date);
11
12 SELECT * FROM sales.orders;
13
14 SELECT
15     order_id,
16     customer_id,
17     order_date,
18     order_status
19 FROM sales.orders
20 WHERE customer_id = 259
21     AND order_date = '2016-01-01';
```

100 % No issues found

Results Messages

	order_id	customer_id	order_date	order_status
1	1	259	2016-01-01	4

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\ujitsu... BikeSto

-- Q3.

-- A teammate suggests adding a unique index on

-- sales.customers(phone_number).

-- What could go wrong with this?

-- What assumption must be true for this to be safe?

-- Hint: Think about duplicate or missing (NULL) values.

Answer:

Technical Context

Unlike standard ANSI SQL, SQL Server treats multiple NULL values as duplicates in a standard UNIQUE index. Therefore, you must verify both existing duplicate numbers and NULL entries before applying the index.

Core Assumption

Business Rule: Every customer's phone number must be strictly unique. No two customers can share the same number, and the current dataset must be completely free of duplicates.

What Could Go Wrong?

Index Creation Fails: If the table already contains duplicate phone numbers, SQL Server will reject and fail to create the UNIQUE index.

Future Operations Blocked: Once active, any attempt to insert or update a record with an existing phone number will trigger an application error and fail the transaction.

The NULL Value Trap: SQL Server allows only one NULL value in a standard unique index. If multiple customers have missing (blank) phone numbers, the second NULL will be rejected as a duplicate, breaking form submissions for empty fields.

-- Q4.

-- Look at the columns below from a sales.orders table.

-- Decide which columns SHOULD have an index and which should NOT.

-- Explain your reasoning for each as a comment.

--

-- order_id (Primary Key)

-- status (only 3 values: Pending, Shipped, Delivered)

-- customer_id (Foreign Key)

-- notes (free text, rarely searched)

-- Your answer here :

-- order_id

-- SHOULD HAVE INDEX (Primary/Clustered Index)

-- Reason: It is a unique identifier for each order.

-- It ensures fast lookup, joins, and enforces uniqueness.

- customer_id
 - SHOULD HAVE INDEX (Non-Clustered Index)
 - Reason: Frequently used in WHERE clause and joins.
 - Helps in retrieving all orders for a specific customer.
- order_date
 - SHOULD HAVE INDEX (Non-Clustered Index)
 - Reason: Used in reporting and range queries (BETWEEN, >, <).
 - Improves performance for date-based filtering.
- (customer_id, order_date)
 - SHOULD HAVE COMPOSITE INDEX (Best Choice)
 - Reason: Most common query pattern filters on both columns.
 - Provides efficient index seek instead of full scan.
- shipped_date
 - CONDITIONAL INDEX (Depends on usage)
 - Reason: Useful for tracking shipped/pending orders.
 - Helps in filtering NULL or date ranges.
- required_date
 - SHOULD NOT HAVE INDEX
 - Reason: Rarely used in filtering and low selectivity.
 - Index would increase storage and reduce performance benefit.

-- order_status

-- SHOULD NOT HAVE INDEX (Generally)

-- Reason: Low selectivity (few repeated values like 1,2,3).

-- Not efficient for indexing unless heavily queried.

-- store_id

-- SHOULD HAVE INDEX

-- Reason: Commonly used for filtering orders by store.

-- Useful in reporting and grouping operations.

-- staff_id

-- SHOULD HAVE INDEX

-- Reason: Used for staff performance tracking and filtering.

-- Improves join and search performance.

-- Q5.

-- Write the command to check existing indexes on production.products.

-- Then describe (as a comment) what the output columns tell you.

-- Hint: Use sp_helpindex.

-- Your answer here:

```

1
2  -- Q5.
3  -- Write the command to check existing indexes on production.products.
4  -- Then describe (as a comment) what the output columns tell you.
5  -- Hint: Use sp_helpindex.
6
7  EXEC sp_helpindex 'production.products';
8  -- Output columns explanation:
9
10 -- index_name → Shows the name of each index on the table
11 -- index_description → Tells type of index (e.g. clustered, nonclustered, unique, primary key)
12 -- index_keys → Shows which column(s) are included in the index (and their order)
13

```

6 No issues found Ln: 13, Ch: 1 TABS CRLF

Results Messages

index_name	index_description	index_keys
PK_products_47027DF570C1D656	clustered, unique, primary key located on PRIMARY	product_id

Purpose: Displays detailed information about all existing indexes on a specific table.

Metadata Provided: Identifies which columns are indexed, their ordering, and the index type (e.g., clustered vs. non-clustered, unique).

Use Case: Crucial for analyzing database structure, optimizing query performance, and troubleshooting indexing issues.

-- =====

-- PART B: STORED PROCEDURES

-- =====

-- Q6.

-- Create a stored procedure called sp_GetCustomerOrders

-- that accepts a @CustomerID parameter and returns all orders
-- for that customer showing: order_id, order_date, order_status.
-- Test it using EXEC after you create it.

-- Your answer here:

2-SQLQuery1....ujitsu (62))*

```
1  -- Q6.
2  -- Create a stored procedure called sp_GetCustomerOrders
3  -- that accepts a @CustomerID parameter and returns all orders
4  -- for that customer showing: order_id, order_date, order_status.
5  -- Test it using EXEC after you create it.
6  -- Q6: Create stored procedure sp_GetCustomerOrders
7
8  CREATE PROCEDURE sp_GetCustomerOrders
9      @CustomerID INT
10 AS
11 BEGIN
12     SELECT
13         order_id,
14         order_date,
15         order_status
16     FROM sales.orders
17     WHERE customer_id = @CustomerID;
18 END;
```

100 %

✓ No issues found

Ln: 8, Ch: 1

Messages

Commands completed successfully.

Completion time: 2026-06-03T12:32:27.5414049+05:00

100 %

✓ No issues found

Ln: 4,

✓ Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM)

DESKTOP-26P7AE7\fujitsu...

BikeSt

2-SQLQuery1....ujitsu (62))*

E7

```
12      SELECT
13          order_id,
14          order_date,
15          order_status
16      FROM sales.orders
17      WHERE customer_id = @CustomerID;
18  END;
19
20  SELECT customer_id FROM sales.orders
21
22  --Test the stored procedure
23  EXEC sp_GetCustomerOrders @CustomerID = 1;
```

100 %

No issues found

Results

Messages

	customer_id
1	1
2	1
3	1
4	2
5	2
6	2
7	3

2-SQLQuery1....ujitsu (62))*

```
12  SELECT
13      order_id,
14      order_date,
15      order_status
16  FROM sales.orders
17  WHERE customer_id = @CustomerID;
18  END;
19
20  SELECT customer_id FROM sales.orders
21
22  --Test the stored procedure
23  EXEC sp_GetCustomerOrders @CustomerID = 1;
```

100 %

✓ No issues found

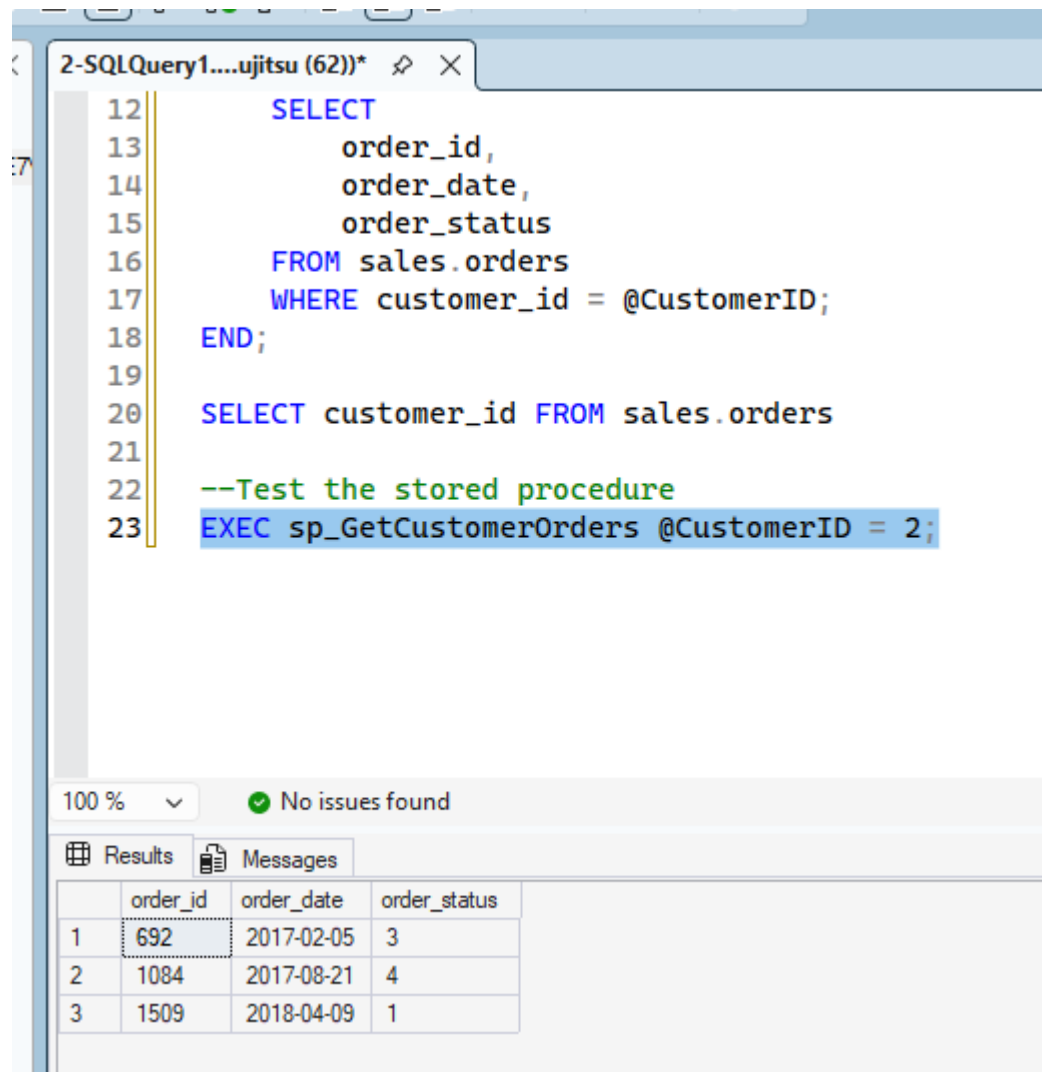
Results

Messages

	order_id	order_date	order_status
1	599	2016-12-09	4
2	1555	2018-04-18	1
3	1613	2018-11-18	3

✓ Query executed successfully.

DESKTOP-26P7AE7



The screenshot shows a SQL Server Enterprise Manager window titled "2-SQLQuery1....ujitsu (62))*". The query editor contains the following SQL code:

```
12      SELECT
13          order_id,
14          order_date,
15          order_status
16      FROM sales.orders
17      WHERE customer_id = @CustomerID;
18  END;
19
20  SELECT customer_id FROM sales.orders
21
22  --Test the stored procedure
23  EXEC sp_GetCustomerOrders @CustomerID = 2;
```

Below the query editor, the "Results" tab is active, displaying a table with 3 rows and 4 columns: order_id, order_date, and order_status. The table contains the following data:

	order_id	order_date	order_status
1	692	2017-02-05	3
2	1084	2017-08-21	4
3	1509	2018-04-09	1

-- Q7.

-- Modify sp_GetCustomerOrders from Q6 so that if no orders

-- are found for the given customer, it returns the message:

-- 'No orders found for this customer'

-- Hint: Use IF EXISTS or check @@ROWCOUNT.

-- Your answer here:

6-SQLQuery1....ujitsu (62))*

```
4  -- 'No orders found for this customer'
5  -- Hint: Use IF EXISTS or check @@ROWCOUNT.
6  -- Q7: Modified stored procedure with no-data message
7
8  ALTER PROCEDURE sp_GetCustomerOrders
9      @CustomerID INT
10 AS
11 BEGIN
12     -- First check if customer has any orders
13     IF EXISTS (
14         SELECT 1
15         FROM sales.orders
16         WHERE customer_id = @CustomerID
17     )
18     BEGIN
19         SELECT
20             order_id,
21             order_date,
22             order_status
23         FROM sales.orders
24         WHERE customer_id = @CustomerID;
25     END
26 ELSE
27     BEGIN
28         SELECT 'No orders found for this customer' AS message;
29     END
30 END;
```

100 %

1 0

6-SQLQuery1.....ujitsu (62))*

```
1  -- Q7.
2  -- Modify sp_GetCustomerOrders from Q6 so that if no orders
3  -- are found for the given customer, it returns the message:
4  -- 'No orders found for this customer'
5  -- Hint: Use IF EXISTS or check @@ROWCOUNT.
6  -- Q7: Modified stored procedure with no-data message
7
8  ALTER PROCEDURE sp_GetCustomerOrders
9      @CustomerID INT
10 AS
11 BEGIN
12     -- First check if customer has any orders
13     IF EXISTS (
14         SELECT 1
15         FROM sales.orders
16         WHERE customer_id = @CustomerID
```

100 %

1 0

Messages

Commands completed successfully.

Completion time: 2026-06-03T12:40:36.3247874+05:00

The screenshot shows a SQL Server Enterprise Manager window with a query titled "6-SQLQuery1....ujitsu (62)*". The query is as follows:

```
23         FROM sales.orders
24         WHERE customer_id = @CustomerID;
25     END
26     ELSE
27     BEGIN
28         SELECT 'No orders found for this customer' AS message
29     END
30 END;
31 --Test it
32 EXEC sp_GetCustomerOrders @CustomerID = 1;
```

Below the query window, the "Results" tab is active, displaying a table with the following data:

	order_id	order_date	order_status
1	599	2016-12-09	4
2	1555	2018-04-18	1
3	1613	2018-11-18	3

At the bottom of the window, a status bar indicates "Query executed successfully." and the server name "DESKTOP-26P7AE7 (17.0 RTM)" is visible.

-- Q8.

-- Create a stored procedure sp_ProductsByCategory that accepts:

-- @CategoryID INT

-- @MaxPrice DECIMAL(10,2) with a default value of 9999

-- It should return all matching products ordered by price (low to high).

-- Hint: Use a default parameter value like you saw with @threshold.

-- Your answer here:

7-SQLQuery1....ujitsu (62))*

```
4  -- @MaxPrice    DECIMAL(10,2) with a default value of 9999
5  -- It should return all matching products ordered by price (low to high).
6  -- Hint: Use a default parameter value like you saw with @threshold.
7
8  -- Stored procedure with default parameter
9
10 CREATE PROCEDURE sp_ProductsByCategory
11     @CategoryID INT,
12     @MaxPrice DECIMAL(10,2) = 9999
13 AS
14 BEGIN
15     SELECT
16         product_id,
17         product_name,
18         list_price,
19         category_id
20     FROM production.products
21     WHERE category_id = @CategoryID
22         AND list_price <= @MaxPrice
23     ORDER BY list_price ASC;
24 END;
```

100 % No issues found

Messages

Completion time: 2026-06-03T12:44:38.1690666+05:00

100 % No issues found

Ln: 4, C

Query executed successfully.

DESKTOP-26P7AE7 (17.0 RTM)

DESKTOP-26P7AE7\ujitsu...

BikeStor

7-SQLQuery1....ujitsu (62))*

```
16         product_id,  
17         product_name,  
18         list_price,  
19         category_id  
20     FROM production.products  
21     WHERE category_id = @CategoryID  
22           AND list_price <= @MaxPrice  
23     ORDER BY list_price ASC;  
24 END;  
25  
26 -- Test it :  
27 -- Using default MaxPrice (9999)  
28 EXEC sp_ProductsByCategory @CategoryID = 1;  
29  
30 -- Custom MaxPrice  
31 EXEC sp_ProductsByCategory @CategoryID = 1, @MaxPrice = 500
```

100 % No issues found

Results Messages

	product_id	product_name	list_price	category_id
1	263	Strider Classic 12 Balance Bike - 2018	89.99	1
2	84	Sun Bicycles Lil Kitt'n - 2017	109.99	1
3	86	Trek Girl's Kickster - 2017	149.99	1
4	83	Trek Boy's Kickster - 2015/2017	149.99	1
5	268	Trek Kickster - 2018	159.99	1
6	87	Trek Precaliber 12 Boys - 2017	189.99	1
7	88	Trek Precaliber 12 Girls - 2017	189.99	1

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTO

7-SQLQuery1....ujitsu (62))*

```

16      product_id,
17      product_name,
18      list_price,
19      category_id
20  FROM production.products
21  WHERE category_id = @CategoryID
22         AND list_price <= @MaxPrice
23  ORDER BY list_price ASC;
24  END;
25
26  -- Test it :
27  -- Using default MaxPrice (9999)
28  EXEC sp_ProductsByCategory @CategoryID = 1;
29
30  -- Custom MaxPrice
31  EXEC sp_ProductsByCategory @CategoryID = 1, @MaxPrice = 500;

```

100 % No issues found Ln:

Results Messages

	product_id	product_name	list_price	category_id
1	263	Strider Classic 12 Balance Bike - 2018	89.99	1
2	84	Sun Bicycles Lil Kitt'n - 2017	109.99	1
3	86	Trek Girl's Kickster - 2017	149.99	1
4	83	Trek Boy's Kickster - 2015/2017	149.99	1
5	268	Trek Kickster - 2018	159.99	1
6	87	Trek Precaliber 12 Boys - 2017	189.99	1
7	88	Trek Precaliber 12 Girls - 2017	189.99	1

Query executed successfully. DESKTOP-26P7AE7 (17.0 RTM) DESKTOP-26P7AE7\fujitsu...

-- =====

-- PART C: MIXED / THINK QUESTIONS

-- =====

-- Q9.

-- You have a sales.orders table with 2 million rows.

-- A stored procedure filters by store_id and order_date.

-- It runs very slowly.

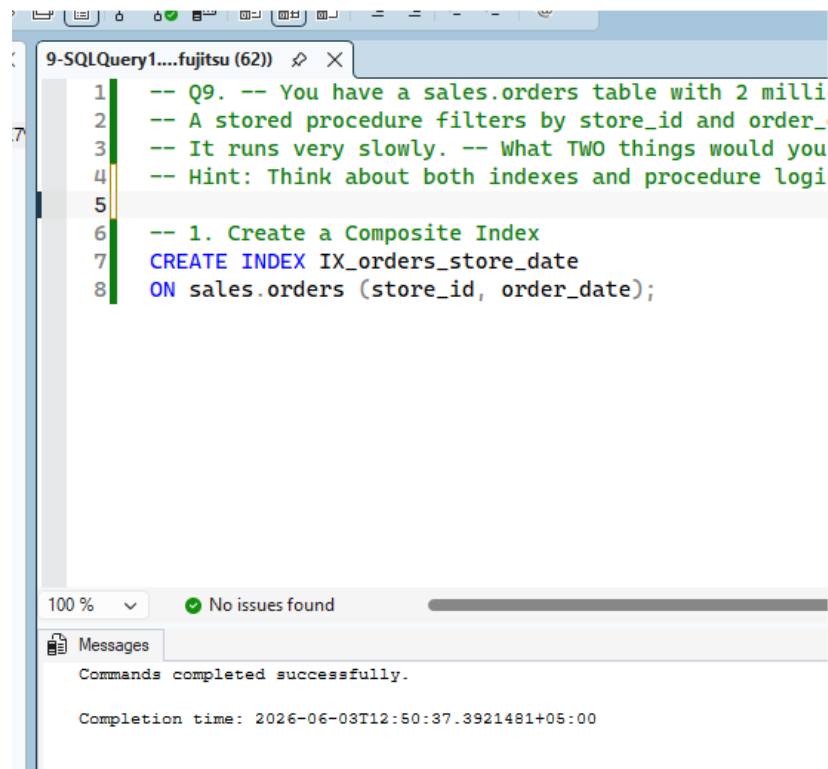
-- What TWO things would you do to fix it, and why?

-- Hint: Think about both indexes and procedure logic.

-- Your answer here (write as a comment):

If a stored procedure is slow on a **2 million row sales.orders table**, especially when filtering by store_id and order_date, I would do the following TWO improvements:

1. Create a Composite Index : Why?



```
9-SQLQuery1....fujitsu (62) X
1  -- Q9. -- You have a sales.orders table with 2 milli
2  -- A stored procedure filters by store_id and order_
3  -- It runs very slowly. -- What TWO things would you
4  -- Hint: Think about both indexes and procedure logi
5
6  -- 1. Create a Composite Index
7  CREATE INDEX IX_orders_store_date
8  ON sales.orders (store_id, order_date);
```

100 % No issues found

Messages

Commands completed successfully.

Completion time: 2026-06-03T12:50:37.3921481+05:00

Query filtering is happening on store_id and order_date

A composite index allows SQL Server to quickly locate matching rows instead of scanning 2 million rows

It reduces full table scans (Table Scan → Index Seek)

-- GOOD (SARGable):

Make sure the WHERE clause is written in a way that uses indexes properly

WHERE store_id = @StoreID

AND order_date >= @StartDate

AND order_date < @EndDate

Why?

Avoids functions like:

YEAR(order_date)

CAST(order_date AS DATE)

Such functions break index usage

Using range queries keeps the query SARGable (Search Argument Able) so index works efficiently

First fix: add a composite index on (store_id, order_date) to speed up data lookup

Second fix: ensure the procedure uses a SARGable WHERE clause so SQL Server can use the index properly

-- Q10.

-- A junior developer creates indexes on EVERY column of a table

-- to "make everything faster".

-- Write a short explanation (3-5 sentences) of why this is

-- actually a bad idea.

-- Hint: Think about how INSERT, UPDATE, and DELETE are affected.

-- Your answer here (write as a comment):

Creating indexes on every column is a bad idea because indexes are not free. While they can speed up SELECT queries, they significantly slow down INSERT, UPDATE, and DELETE operations because every index must also be updated whenever data changes. This increases storage usage and can lead to

unnecessary memory and disk overhead. Too many indexes can also confuse the query optimizer and reduce overall performance instead of improving it. In practice, indexes should only be created on columns that are frequently used in WHERE, JOIN, or ORDER BY clauses.

-- =====

-- END OF HOMEWORK

-- =====